EXPERIMENT- 5

Student Name: Gursharan Singh	**Semester:** 6th
UID: 23BAI70554	**Date of Performance:** 18/02/2026
Branch: B.E. CSE (AI & ML)	**Subject:** Full Stack – II
Class: 23AIT_KRG – G1	**Subject Code:** 23CSH-382

• Aim:-

To verify the correctness and reliability of the EcoTrack React application by writing automated tests using Jest and React Testing Library, and by analyzing application behavior using debugging tools.

• Objectives:-

After completing this experiment, the student will be able to:-
1. Understand the purpose of automated testing in frontend applications
2. Write unit tests for JavaScript utility functions using Jest
3. Use different Jest matchers to validate expected outputs and behaviors
4. Test React components using React Testing Library
5. Verify UI rendering by querying elements from the DOM
6. Implement asynchronous testing using findBy and waitFor methods
7. Apply mocking to simulate API or external data responses in tests
8. Perform snapshot testing to detect unintended UI changes
9. Debug failing tests and application logic using browser Developer Tools and breakpoints
10. Analyze application behavior and errors systematically rather than manual checking

• Implementation/Code:-

1) App.Integration.test.jsx:-

```
42  describe('App Integration Tests', () => {
120   test('suspense fallback works for lazy components', async () => {
124       </TestWrapper>
125     );
126
127     // The app should handle Suspense properly
128     await waitFor(() => {
129       expect(screen.getByText('ECO TRACK')).toBeInTheDocument();
130     });
131   });
132
133   test('global styles and CSS modules are applied', () => {
134     render(
135       <TestWrapper>
136         <App />
137       </TestWrapper>
138     );
139
140     // Check that CSS classes are applied (this is a basic check)
141     const header = screen.getByRole('banner');
142     expect(header).toBeInTheDocument();
143   });
144
145   test('app structure is consistent', () => {
146     const { container } = render(
147       <TestWrapper>
148         <App />
149       </TestWrapper>
150     );
151
152     // Check that the main app structure exists
153     expect(container.firstChild).toBeInTheDocument();
154     expect(container.querySelector('header')).toBeInTheDocument();
155   });
156 });
```

```jsx
// Mock the logsSlice for testing
jest.mock('../store/logsSlice', () => ({
  ...jest.requireActual('../store/logsSlice'),
  fetchLogs: jest.fn(),
}));

const createMockStore = (initialState = {}) => {
  return configureStore({
    reducer: {
      logs: logsSlice,
    },
    preloadedState: {
      logs: {
        data: [],
        loading: false,
        error: null,
        ...initialState.logs,
      },
    },
  });
};

const TestWrapper = ({ children, initialState }) => (
  <Provider store={createMockStore(initialState)}>
    <BrowserRouter>
      <AuthProvider>
        {children}
      </AuthProvider>
    </BrowserRouter>
  </Provider>
);

describe('App Integration Tests', () => {
  test('renders app without crashing', () => {
```

2) Header.snapshot.test.jsx:-

FSD > Shreshta > Exp5 > src > components > __tests__ > Header.snapshot.test.jsx > ...

```jsx
import React from 'react';
import { render, screen } from '@testing-library/react';
import { BrowserRouter } from 'react-router-dom';
import { AuthProvider } from '../context/auth';
import Header from '../Header';

const TestWrapper = ({ children }) => (
  <BrowserRouter>
    <AuthProvider>
      {children}
    </AuthProvider>
  </BrowserRouter>
);

describe('Header Component Snapshots', () => {
  test('matches snapshot when not authenticated', () => {
    const { asFragment } = render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    expect(asFragment()).toMatchSnapshot();
  });

  test('matches snapshot structure', () => {
    const { container } = render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    expect(container.firstChild).toMatchSnapshot();
  });

  test('has consistent DOM structure', () => {
    render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    // Check that the main structure is consistent
    const header = screen.getByRole('banner');
    expect(header).toBeInTheDocument();
    expect(header.querySelector('h1')).toBeInTheDocument();
    expect(header.querySelector('nav')).toBeInTheDocument();
  });
});
```

3) Header.test.jsx:-

FSD > Shreshta > Exp5 > src > components > __tests__ > Header.test.jsx > ...

```
// Wrapper component to provide both router and auth context
const TestWrapper = ({ children }) => (
  <BrowserRouter>
    <AuthProvider>
      {children}
    </AuthProvider>
  </BrowserRouter>
);

describe('Header Component', () => {
  test('renders ECO TRACK title', () => {
    render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    expect(screen.getByText('ECO TRACK')).toBeInTheDocument();
  });

  test('shows login link when not authenticated', () => {
    render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    const loginLink = screen.getByText('Login');
    expect(loginLink).toBeInTheDocument();
    expect(loginLink.closest('a')).toHaveAttribute('href', '/login');
  });

  test('shows navigation links when authenticated', async () => {
    render(
      <TestWrapper>
        <Header />
      </TestWrapper>
    );

    // Simulate authentication by setting the context directly
    // This would require updating the AuthProvider to allow test overrides
    // For now, let's test the logout functionality

    const loginLink = screen.getByText('Login');
    expect(loginLink).toBeInTheDocument();
  });

  test('renders navigation structure correctly', () => {
    render(
      <TestWrapper>
        <Header />
      </TestWrapper>
```

4) Login.test.jsx:-

FSD > Shreshta > Exp5 > src > pages > __tests__ > Login.test.jsx > ...

```
const TestWrapper = ({ children }) => (
  <BrowserRouter>
    <AuthProvider>
      {children}
    </AuthProvider>
  </BrowserRouter>
);

describe('Login Component', () => {
  test('renders login form', () => {
    render(
      <TestWrapper>
        <Login />
      </TestWrapper>
    );

    expect(screen.getByRole('heading', { name: /login/i })).toBeInTheDocument();
    expect(screen.getByLabelText(/username/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /login/i })).toBeInTheDocument();
  });

  test('shows validation errors for empty fields', async () => {
    const user = userEvent.setup();

    render(
      <TestWrapper>
        <Login />
      </TestWrapper>
    );

    const loginButton = screen.getByRole('button', { name: /login/i });
    await user.click(loginButton);

    // Wait for validation messages
    await waitFor(() => {
      expect(screen.getByText(/username is required/i)).toBeInTheDocument();
      expect(screen.getByText(/password is required/i)).toBeInTheDocument();
    });
  });

  test('allows user to type in form fields', async () => {
    const user = userEvent.setup();

    render(
      <TestWrapper>
        <Login />
      </TestWrapper>
    );

    const usernameInput = screen.getByLabelText(/username/i);
    const passwordInput = screen.getByLabelText(/password/i);
```

5) Logs.test.jsx:-

FSD > Shreshta > Exp5 > src > pages > __tests__ > Logs.test.jsx > ...

```
jest.mock('../../store/logsSlice', () => ({
  fetchLogs: jest.fn(),
}));

// Mock data
const mockLogs = [
  { id: 1, activity: 'Car Travel', carbon: 4 },
  { id: 2, activity: 'Electricity Usage', carbon: 6 },
  { id: 3, activity: 'Cycling', carbon: 0 },
];

const createMockStore = (initialState) => {
  return configureStore({
    reducer: {
      logs: logsSlice,
    },
    preloadedState: initialState,
  });
};

const TestWrapper = ({ children, storeState }) => (
  <Provider store={createMockStore(storeState)}>
    <BrowserRouter>
      <AuthProvider>
        {children}
      </AuthProvider>
    </BrowserRouter>
  </Provider>
);

describe('Logs Component', () => {
  let mockDispatch;

  beforeEach(() => {
    mockDispatch = jest.fn();
    jest.clearAllMocks();
  });

  test('renders loading state', () => {
    const storeState = {
      logs: {
        data: [],
        loading: true,
        error: null,
      },
    };

    render(
      <TestWrapper storeState={storeState}>
        <Logs />
      </TestWrapper>
    );
```

6) logsSlice.test.js:-

FSD > Shreshta > Exp5 > src > store > __tests__ > logsSlice.test.js > ...

```javascript
const mockLogs = [
  { id: 1, activity: "Car Travel", carbon: 4 },
  { id: 2, activity: "Electricity Usage", carbon: 6 },
  { id: 3, activity: "Cycling", carbon: 0 },
];

// Mock the logs data module
jest.mock('../../data/logs', () => mockLogs);

describe('Logs Slice', () => {
  let store;

  beforeEach(() => {
    store = configureStore({
      reducer: {
        logs: logsSlice,
      },
    });
    jest.clearAllMocks();
  });

  test('should return the initial state', () => {
    const initialState = store.getState().logs;
    expect(initialState).toEqual({
      data: [],
      loading: false,
      error: null,
    });
  });

  test('should handle fetchLogs.pending', () => {
    store.dispatch(fetchLogs.pending());
    const state = store.getState().logs;

    expect(state.loading).toBe(true);
    expect(state.error).toBe(null);
  });

  test('should handle fetchLogs.fulfilled', () => {
    store.dispatch(fetchLogs.fulfilled(mockLogs));
    const state = store.getState().logs;

    expect(state.loading).toBe(false);
    expect(state.data).toEqual(mockLogs);
    expect(state.error).toBe(null);
  });

  test('should handle fetchLogs.rejected', () => {
    const mockError = 'Failed to fetch logs';
    store.dispatch(fetchLogs.rejected(mockError));
    const state = store.getState().logs;
```

7) carbonCalculations.test.js:-

FSD > Shreshta > Exp5 > src > utils > __tests__ > carbonCalculations.test.js > ...

```javascript
import {
  calculateCarbonFootprint,
  calculateTotalCarbon,
  getCarbonGrade,
  formatCarbonValue
} from '../carbonCalculations';

describe('Carbon Calculation Utilities', () => {
  describe('calculateCarbonFootprint', () => {
    test('should calculate carbon footprint for car travel', () => {
      const result = calculateCarbonFootprint('commute', 10, 'car');
      expect(result).toBe(2.1);
    });

    test('should calculate carbon footprint for bus travel', () => {
      const result = calculateCarbonFootprint('commute', 10, 'bus');
      expect(result).toBe(0.89);
    });

    test('should calculate carbon footprint for train travel', () => {
      const result = calculateCarbonFootprint('commute', 10, 'train');
      expect(result).toBe(0.41);
    });

    test('should return 0 for bike travel', () => {
      const result = calculateCarbonFootprint('commute', 10, 'bike');
      expect(result).toBe(0);
    });

    test('should return 0 for walking', () => {
      const result = calculateCarbonFootprint('commute', 10, 'walk');
      expect(result).toBe(0);
    });

    test('should handle unknown transport mode', () => {
      const result = calculateCarbonFootprint('commute', 10, 'spaceship');
      expect(result).toBe(0);
    });

    test('should round to 2 decimal places', () => {
      const result = calculateCarbonFootprint('commute', 7, 'car');
      expect(result).toBe(1.47);
    });

    test('should handle zero distance', () => {
      const result = calculateCarbonFootprint('commute', 0, 'car');
      expect(result).toBe(0);
    });
  });

  describe('calculateTotalCarbon', () => {
    test('should sum carbon values from logs', () => {
```

- Outputs:-

1) App integration tests:-

```
> exp5@0.0.0 test
> jest src/__tests__/App.integration.test.jsx

● Validation Warning:

  Unknown option "moduleNameMapping" with value {"\\.(css|less|scss|sass)$": "identity-obj-proxy"} was found.
  This is probably a typing mistake. Fixing it will remove this message.

  Configuration Documentation:
  https://jestjs.io/docs/configuration

● Validation Warning:

  Unknown option "moduleNameMapping" with value {"\\.(css|less|scss|sass)$": "identity-obj-proxy"} was found.
  This is probably a typing mistake. Fixing it will remove this message.

  Configuration Documentation:
  https://jestjs.io/docs/configuration
```

```
PASS src/__tests__/App.integration.test.jsx
  App Integration Tests
    √ renders app without crashing (85 ms)
    √ shows login page by default when not authenticated (56 ms)
    √ navigation structure is present (19 ms)
    √ app handles route changes (11 ms)
    √ lazy loaded components are handled (16 ms)
    √ error boundaries are handled gracefully (6 ms)
    √ suspense fallback works for lazy components (12 ms)
    √ global styles and CSS modules are applied (12 ms)
    √ app structure is consistent (11 ms)

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        2.271 s
Ran all test suites matching src/__tests__/App.integration.test.jsx.
```

2) Carbon calculation tests:-

```
> exp5@0.0.0 test
> jest src/utils/__tests__/carbonCalculations.test.js src/__tests__/App.integration.test.jsx

● Validation Warning:

  Unknown option "moduleNameMapping" with value {"\\.(css|less|scss|sass)$": "identity-obj-proxy"} was found.
  This is probably a typing mistake. Fixing it will remove this message.

  Configuration Documentation:
  https://jestjs.io/docs/configuration

● Validation Warning:

  Unknown option "moduleNameMapping" with value {"\\.(css|less|scss|sass)$": "identity-obj-proxy"} was found.
  This is probably a typing mistake. Fixing it will remove this message.

  Configuration Documentation:
  https://jestjs.io/docs/configuration

PASS src/utils/__tests__/carbonCalculations.test.js
PASS src/__tests__/App.integration.test.jsx
  ● Console
```

```
Test Suites: 2 passed, 2 total
Tests:       30 passed, 30 total
Snapshots:   0 total
Time:        2.736 s
Ran all test suites matching src/utils/__tests__/carbonCalculations.test.js|src/__tests__/App.integration.test.jsx.
```

• <u>Learning Outcomes:-</u>

1. Developed a strong understanding of automated testing by implementing Jest and React Testing Library to ensure application correctness, reliability, and regression prevention.
2. Gained hands-on experience in writing unit, component, and integration tests, including edge case handling and validation of React UI behavior.
3. Learned to test asynchronous operations, mock external dependencies, and Redux stores to achieve isolated, deterministic, and scalable test cases.
4. Acquired practical debugging skills by systematically identifying and resolving common testing issues related to environment setup, routing, and context usage.
5. Understood test coverage analysis and documentation practices by enforcing 70% coverage thresholds and maintaining structured testing and debugging guides.